

```
#부동산 투자 예측 모델 구현 - 데이터 병합
# AI테크노융합학과
# 유영재, 윤지환
from google.colab import drive
drive.mount('/content/drive')
```

↩ Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
# prompt: 드라이브의 estate 폴더의 "_샘플" csv 리스트 파일을 보여줘
import unicodedata
import os
```

```
# Define the path to the folder in Google Drive
folder_path = '/content/drive/My Drive/estate'
```

```
# List files in the folder
files = os.listdir(folder_path)
```

```
print(files)
```

↩ ['estate_org.csv', '.ipynb_checkpoints', 'school_org.csv', 'school.csv', 'subway_org.csv', 'subway.csv', 'estate.csv']

```
import pandas as pd
import os
```

```
# Load the dataframes
estate_df = pd.read_csv(os.path.join(folder_path, 'estate_org.csv'))
subway_df = pd.read_csv(os.path.join(folder_path, 'subway_org.csv'))
```

```
# Normalize column names to remove whitespace
estate_df.columns = [col.strip() for col in estate_df.columns]
subway_df.columns = [col.strip() for col in subway_df.columns]
```

```
# Debug: Print columns and dtypes
print("Estate columns:", estate_df.columns.tolist())
print("Subway columns:", subway_df.columns.tolist())
print("Subway data types:", subway_df.dtypes)
print("First few rows of subway_df:")
print(subway_df.head())
```

```
# Check if we need to rename columns for consistency
if '자치구명' in subway_df.columns:
    subway_df = subway_df.rename(columns={'자치구명': '자치구'})
    print("Renamed '자치구명' to '자치구'")
```

```
# Count number of stations per 자치구
# Method 1: If the data shows one row per station
if '역개수' not in subway_df.columns:
    # Count rows per district (each row represents a station)
    subway_counts = subway_df.groupby('자치구').size().reset_index(name='역개수')
    print("Created subway counts by counting rows per district")
else:
    # Method 2: If '역개수' column already exists, sum it up
    subway_counts = subway_df.groupby('자치구')['역개수'].sum().reset_index()
    print("Used existing '역개수' column")
```

```
print("Subway counts:")
print(subway_counts)
```

```
# Check estate_df columns for merging
print("Estate columns for merging:", estate_df.columns.tolist())
```

```
# estate_df에 이미 '역개수' 컬럼이 있는지 확인
if '역개수' in estate_df.columns:
    print("⚠ estate_df에 이미 '역개수' 컬럼이 존재합니다. 기존 컬럼을 삭제하고 새로 병합합니다.")
    estate_df = estate_df.drop(columns=['역개수'])
```

```
# 자치구명을 사용해서 병합
estate_district_col = '자치구명'
```

```
if estate_district_col not in estate_df.columns:
    print("❌ estate_df에 '자치구명' 컬럼을 찾을 수 없습니다")
    print("사용 가능한 컬럼:", estate_df.columns.tolist())
else:
    # subway_counts의 자치구명을 subway_counts의 컬럼명에 맞게 변경
    subway_counts = subway_counts.rename(columns={'자치구': 'subway_자치구'})

    # estate_df와 subway_counts 병합
    merged_df = pd.merge(estate_df, subway_counts,
```

```
left_on=estate_district_col, right_on='subway_자치구', how='left')
```

```
# 불필요한 컬럼 제거
```

```
merged_df = merged_df.drop(columns=['subway_자치구'])
```

```
# NaN 값을 0으로 채우기 (지하철역이 없는 자치구)
```

```
merged_df['역개수'] = merged_df['역개수'].fillna(0).astype(int)
```

```
# temp.csv로 저장
```

```
merged_df.to_csv(os.path.join(folder_path, 'temp.csv'), index=False)
```

```
print("✅ temp.csv 파일이 성공적으로 생성되었습니다.")
```

```
print(merged_df.head())
```

```
# 지하철역이 있는 자치구 정보 표시
```

```
districts_with_stations = merged_df[merged_df['역개수'] > 0][['자치구명', '역개수']].drop_duplicates().sort_values('역개수', ascending=False)
```

```
<ipython-input-49-2325970252>:5: DtypeWarning: Columns (7,8) have mixed types. Specify dtype option on import or set low_memory=False.
estate_df = pd.read_csv(os.path.join(folder_path, 'estate_org.csv'))
Estate columns: ['접수연도', '자치구코드', '자치구명', '법정동코드', '법정동명', '지번구분', '지번구분명', '본번', '부번', '건물명', '계약일',
Subway columns: ['자치구', '해당역(호선)', '역개수']
Subway data types: 자치구      object
                    해당역(호선) object
                    역개수      int64
dtype: object
First few rows of subway_df:
   자치구      해당역(호선)  역개수
0  강남구  삼성(2), 선릉(2), 역삼(2), 강남(2), 압구정(3), 신사(3), 매봉(...   21
1  강동구  천호(5), 강동(5), 길동(5), 굽은다리(5), 명일(5), 고덕(5), 상일...   15
2  강북구              수유(4), 미아(4), 미아사거리(4)      3
3  강서구  방화(5), 개화산(5), 김포공항(5), 송정(5), 마곡(5), 발산(5), 우...   9
4  관악구  낙성대(2), 서울대입구(2), 봉천(2), 신림(2)      4
Used existing '역개수' column
Subway counts:
   자치구  역개수
0  강남구     21
1  강동구     15
2  강북구      3
3  강서구      9
4  고양시      1
5  관악구      4
6  광명시      2
7  광진구     11
8  구로구      7
9  금천구      1
10 노원구     13
11 도봉구      3
12 동대문구      6
13 동작구     12
14 마포구     16
15 서대문구      5
16 서초구     12
17 성남시      7
18 성동구     14
19 성북구      9
20 송파구     28
21 양천구      5
22 영등포구     11
23 용산구     10
24 은평구     13
25 의정부시      1
26 종로구     15
27 중구      23
28 중랑구      8
29 하남시      4
Estate columns for merging: ['접수연도', '자치구코드', '자치구명', '법정동코드', '법정동명', '지번구분', '지번구분명', '본번', '부번', '건물명',
✅ temp.csv 파일이 성공적으로 생성되었습니다.
접수연도  자치구코드  자치구명  법정동코드  법정동명  지번구분  지번구분명  본번  부번  건물명  ...  W
0  2024  11230  동대문구  10400  전농동  1.0  대지  103.0  50.0  (103-50)  ...
1  2024  11680  강남구  10100  역삼동  1.0  대지  736.0  24.0  엘지역상에클라트  ...
2  2024  11230  동대문구  10500  답십리동  1.0  대지  994.0  0.0  답십리한화  ...
3  2024  11545  금천구  10300  시흥동  1.0  대지  942.0  2.0  아르떼빌  ...
4  2024  11350  노원구  10500  상계동  1.0  대지  1285.0  0.0  동아불암  ...
```

```
# prompt: temp.csv에서 권리구분, 취소일, 건물용도, 신고구분, 신고한 개업공인중개사 시군구명, 지번구분명, 본번, 부번, 지번구분 컬럼은 제거해주고 같은
# 같은 폴더에 만들어줘
```

```
# Define the columns to drop
```

```
columns_to_drop = [
```

```
    '권리구분',
```

```
    '취소일',
```

```
    '건물용도',
```

```
    '신고구분',
```

```
    '신고한 개업공인중개사 시군구명',
```

```
    '지번구분명',
```

```

'본번',
'부번',
'지번구분'
]

# Load the temp.csv file
try:
    temp_df = pd.read_csv(os.path.join(folder_path, 'temp.csv'))

    # Drop the specified columns if they exist
    # Use errors='ignore' to avoid error if a column doesn't exist
    temp_df = temp_df.drop(columns=columns_to_drop, errors='ignore')

    # Save the modified DataFrame back to temp.csv in the same directory
    temp_df.to_csv(os.path.join(folder_path, 'temp.csv'), index=False)

    print("✅ temp.csv 파일에서 지정된 컬럼이 성공적으로 제거되었습니다.")

except FileNotFoundError:
    print("❌ temp.csv 파일을 찾을 수 없습니다.")
except Exception as e:
    print(f"❌ 파일을 처리하는 중 오류가 발생했습니다: {e}")

```

↻ <ipython-input-50-4176795207>:19: DtypeWarning: Columns (7,8) have mixed types. Specify dtype option on import or set low_memory=False.

```
temp_df = pd.read_csv(os.path.join(folder_path, 'temp.csv'))
```

✅ temp.csv 파일에서 지정된 컬럼이 성공적으로 제거되었습니다.

```

# prompt: school.csv 에는 "도로명주소"와 "도로명상세주소" 가 있어.
# 1. 도로명 주소에서는 "구"만 남겨놓고 다 제거해줘
# 2. 도로명상세주소에서는 "동"만 남겨놓고 다 제거해줘
# "구"와"동"을 그룹바이 해주고 합계를 숫자로 표현해주고 파일을 temp2.csv파일로 만들어줘

```

```

# Ensure pandas is imported
import pandas as pd

```

```

# Load the school.csv file
try:
    school_df = pd.read_csv(os.path.join(folder_path, 'school_org.csv'))
except FileNotFoundError:
    print("Error: school.csv not found. Please ensure it exists in your mounted Drive folder.")
    exit()

```

```

# Normalize column names
school_df.columns = [col.strip() for col in school_df.columns]

```

```

# 1. Process "도로명주소" to keep only "구"
if '도로명주소' in school_df.columns:
    # Use regex to extract the part before "구" and append "구"
    # This assumes the format is "XX시 XX구 ..."
    school_df['구'] = school_df['도로명주소'].str.extract(r'(WS+구)')
    print("Processed '도로명주소' to extract '구'")
else:
    print("Warning: '도로명주소' column not found in school.csv")
    school_df['구'] = None # Add the column as None if not found

```

```

# 2. Process "도로명상세주소" to keep only "동"
if '도로명상세주소' in school_df.columns:
    # Use regex to extract the part before "동" and append "동"
    # This assumes the format is "... XX동 ..."
    school_df['동'] = school_df['도로명상세주소'].str.extract(r'(WS+동)')
    school_df['동'] = school_df['동'].str.replace(' ', '', regex=False)
    print("Processed '도로명상세주소' to extract '동'")
else:
    print("Warning: '도로명상세주소' column not found in school.csv")
    school_df['동'] = None # Add the column as None if not found

```

```

# Convert relevant columns to numeric, coercing errors
# Assuming '합계' refers to some quantitative column you want to sum.
# If there isn't a column named '합계' that needs summing, you might just want to count entries.
# Let's assume there's a column named '합계' or we just want to count rows.
# If you need to sum a specific column, replace 'AnyNumericColumnName' with the actual column name.
numeric_col_to_sum = None # Set this to the actual column name if you need to sum

```

```

# Attempt to find a suitable numeric column if '합계' isn't available or clear
potential_numeric_cols = ['학생수', '교원수', '면적'] # Example potential column names
for col in potential_numeric_cols:
    if col in school_df.columns:
        numeric_col_to_sum = col
        print(f"Using '{numeric_col_to_sum}' for aggregation.")
        break

```

```
# If no specific numeric column is identified for summing, we'll just count entries per group.
if numeric_col_to_sum is None:
    print("No specific numeric column found for summing. Will count entries per '구', '동' group.")
    # Group by '구' and '동' and count the occurrences
    grouped_data = school_df.groupby(['구', '동']).size().reset_index(name='학교수')
    print("Grouped by '구', '동' and counted entries.")
else:
    # Convert the identified numeric column to numeric, handle errors
    school_df[numeric_col_to_sum] = pd.to_numeric(school_df[numeric_col_to_sum], errors='coerce').fillna(0)
    print(f"Converted '{numeric_col_to_sum}' to numeric.")

    # Group by '구' and '동' and sum the specified numeric column
    grouped_data = school_df.groupby(['구', '동'])[numeric_col_to_sum].sum().reset_index(name=f'총_{numeric_col_to_sum}')
    print(f"Grouped by '구', '동' and summed '{numeric_col_to_sum}'.")

# Save the result to temp2.csv
output_path = os.path.join(folder_path, 'temp2.csv') # Save in the same drive folder
grouped_data.to_csv(output_path, index=False)
```

```
print(f"✅ temp2.csv file successfully created at {output_path}")
print("Grouped data sample:")
print(grouped_data.head())
print("Grouped data size:", grouped_data.shape)
```

```
➡ Processed '도로명주소' to extract '구'
Processed '도로명상세주소' to extract '동'
No specific numeric column found for summing. Will count entries per '구', '동' group.
Grouped by '구', '동' and counted entries.
✅ temp2.csv file successfully created at /content/drive/My Drive/estate/temp2.csv
Grouped data sample:
   구   동   학교수
0 강남구 개포동   35
1 강남구 논현동    2
2 강남구 대치동   37
3 강남구 도곡동   18
4 강남구 삼성동    8
Grouped data size: (272, 3)
```

```
# prompt: temp.csv와 temp2.csv 파일을 병합해줘
# 1. "자치구명"과 "구" + "법정동명"과 "동"이 매칭되어야해.
```

```
# Ensure temp_df is loaded correctly (assuming it was saved to folder_path/temp.csv)
temp_df_path = os.path.join(folder_path, 'temp.csv')
temp2_df_path = os.path.join(folder_path, 'temp2.csv')
```

```
try:
    temp_df = pd.read_csv(temp_df_path)
    print(f"✅ Successfully loaded {temp_df_path}")
    print("temp_df columns:", temp_df.columns.tolist())
    print("temp_df sample:")
    print(temp_df.head())

    temp2_df = pd.read_csv(temp2_df_path)
    print(f"✅ Successfully loaded {temp2_df_path}")
    print("temp2_df columns:", temp2_df.columns.tolist())
    print("temp2_df sample:")
    print(temp2_df.head())
```

```
except FileNotFoundError as e:
    print(f"❌ 파일을 찾을 수 없습니다: {e}")
    exit()
```

```
except Exception as e:
    print(f"❌ 파일 로딩 중 오류가 발생했습니다: {e}")
    exit()
```

```
# Merge condition: "자치구명" (from temp_df) with "구" (from temp2_df)
# AND "법정동명" (from temp_df) with "동" (from temp2_df)
```

```
# Check required columns in temp_df
required_temp_cols = ['자치구명', '법정동명']
for col in required_temp_cols:
    if col not in temp_df.columns:
        print(f"❌ temp_df에 '{col}' 컬럼이 없습니다. 병합을 진행할 수 없습니다.")
        print("temp_df 사용 가능한 컬럼:", temp_df.columns.tolist())
        exit()
```

```
# Check required columns in temp2_df
required_temp2_cols = ['구', '동']
# Determine the aggregation column name in temp2_df
# It could be '학교수' or something like '총_학생수'
# We need to find the non-key column in temp2_df
temp2_value_col = None
for col in temp2_df.columns:
```

```

if col not in ['구', '동']:
    temp2_value_col = col
    break

if not all(col in temp2_df.columns for col in required_temp2_cols) or temp2_value_col is None:
    print(f"❌ temp2_df에 '{required_temp2_cols}' 컬럼 또는 집계된 값이 없습니다. 병합을 진행할 수 없습니다.")
    print("temp2_df 사용 가능한 컬럼:", temp2_df.columns.tolist())
    exit()

print(f"\n✅ 병합 조건:")
print(f" - temp_df['자치구명'] == temp2_df['구']")
print(f" - temp_df['법정동명'] == temp2_df['동']")
print(f" - 병합할 컬럼: temp2_df['{temp2_value_col}']")

# Perform the merge
merged_school_df = pd.merge(
    temp_df,
    temp2_df,
    left_on=['자치구명', '법정동명'],
    right_on=['구', '동'],
    how='left' # Use left merge to keep all rows from temp_df
)

# Remove the duplicate '구' and '동' columns from temp2_df after merging
merged_school_df = merged_school_df.drop(columns=['구', '동'], errors='ignore')

# Fill NaN values in the merged column with 0 (for locations with no matching entry in temp2.csv)
merged_school_df[temp2_value_col] = merged_school_df[temp2_value_col].fillna(0).astype(int)

# Define the output file path
final_merged_output_path = os.path.join(folder_path, 'merged_with_school.csv') # Or temp3.csv, etc.

# Save the final merged dataframe
merged_school_df.to_csv(final_merged_output_path, index=False)

print(f"\n✅ temp.csv와 temp2.csv 파일이 성공적으로 병합되었습니다.")
print(f" 결과 파일: {final_merged_output_path}")
print("최종 병합된 데이터 크기:", merged_school_df.shape)
print("최종 병합된 데이터 컬럼:")
print(merged_school_df.columns.tolist())
print("\n최종 병합된 데이터 샘플:")
print(merged_school_df.head())

# Optional: Display rows where the new school column is > 0
rows_with_school_data = merged_school_df[merged_school_df[temp2_value_col] > 0]
if not rows_with_school_data.empty:
    print(f"\n학교 정보가 병합된 행 샘플 (where {temp2_value_col} > 0):")
    print(rows_with_school_data[['자치구명', '법정동명', temp2_value_col]].head())
else:
    print("\n병합된 데이터 중 학교 정보가 있는 행이 없습니다.")

🔄 ✅ Successfully loaded /content/drive/My Drive/estate/temp.csv
temp_df columns: ['접수연도', '자치구코드', '자치구명', '법정동코드', '법정동명', '건물명', '계약일', '물건금액(만원)', '건물면적(㎡)', '토지면적(㎡)']
temp_df sample:
  접수연도  자치구코드  자치구명  법정동코드  법정동명      건물명      계약일  물건금액(만원)  건물면적(㎡)  W
0  2024    11230    동대문구  10400    전농동  (103-50)  20241231      24000      44.04
1  2024    11680    강남구    10100    역삼동  엘지역삼에클라트  20241231      19000      28.64
2  2024    11230    동대문구  10500    답십리동  답십리한화  20241231      91500      84.86
3  2024    11545    금천구    10300    시흥동  아르떼빌  20241231      27000      25.28
4  2024    11350    노원구    10500    상계동  동아불암  20241231      70000      114.24

  토지면적(㎡)  총      건축년도  역개수
0    20.00  -1.0  1991.0    6
1    40.01  11.0  2004.0   21
2     0.00  13.0  2001.0    6
3    18.00   5.0  2021.0    1
4     0.00   4.0  1999.0   13

✅ Successfully loaded /content/drive/My Drive/estate/temp2.csv
temp2_df columns: ['구', '동', '학교수']
temp2_df sample:
   구  동  학교수
0 강남구  개포동    35
1 강남구  논현동     2
2 강남구  대치동    37
3 강남구  도곡동    18
4 강남구  삼성동     8

✅ 병합 조건:
- temp_df['자치구명'] == temp2_df['구']
- temp_df['법정동명'] == temp2_df['동']
- 병합할 컬럼: temp2_df['학교수']

```

temp.csv와 temp2.csv 파일이 성공적으로 병합되었습니다.
결과 파일: /content/drive/My Drive/estate/merged_with_school.csv

최종 병합된 데이터 크기: (96787, 14)

최종 병합된 데이터 컬럼:
['접수연도', '자치구코드', '자치구명', '법정동코드', '법정동명', '건물명', '계약일', '물건금액(만원)', '건물면적(㎡)', '토지면적(㎡)', '층', '학교수']

최종 병합된 데이터 샘플:

	접수연도	자치구코드	자치구명	법정동코드	법정동명	건물명	계약일	물건금액(만원)	건물면적(㎡)	W
0	2024	11230	동대문구	10400	전농동	(103-50)	20241231	24000	44.04	
1	2024	11680	강남구	10100	역삼동	엘지역삼에클라트	20241231	19000	28.64	
2	2024	11230	동대문구	10500	답십리동	답십리한화	20241231	91500	84.86	
3	2024	11545	금천구	10300	시흥동	아르떼빌	20241231	27000	25.28	
4	2024	11350	노원구	10500	상계동	동아불암	20241231	70000	114.24	

	토지면적(㎡)	층	건축년도	역개수	학교수
0	20.00	-1.0	1991.0	6	28
1	40.01	11.0	2004.0	21	4
2	0.00	13.0	2001.0	6	3
3	18.00	5.0	2021.0	1	57
4	0.00	4.0	1999.0	13	48

학교 정보가 병합된 행 샘플 (where 학교수 > 0):

	자치구명	법정동명	학교수
0	동대문구	전농동	28
1	강남구	역삼동	4